Avanti Gulhane

November 11, 2026

Introduction to Programming with Python

Assignment 05 – Advanced Collections and Error Handling

Introduction

In Module 05, I learned how to organize information using dictionaries, store structured records in JSON files, and handle errors using try-except statements. I continued developing the course registration program from Assignment 04. Instead of using a two-dimensional list saved in a CSV file, Assignment 05 uses dictionaries and a JSON file to manage student registrations. I used the Module 05 notes and tested the program in PyCharm.

Learning the Basics

Dictionaries and Lists of Dictionaries

A dictionary stores data as key-value pairs. Unlike a list, which accesses elements using numerical indexes, a dictionary accesses values using named keys. For example, a student registration can contain the keys FirstName, LastName, and CourseName. Dictionary keys are case-sensitive. A list of dictionaries can hold multiple registrations, with each dictionary representing one student.

```
student_data = {"FirstName": "John", "LastName": "Smith", "CourseName": "Python 100"}
students = []
students.append(student_data)
print(student_data["FirstName"])
```

Figure 1.1. A dictionary represents a student record, and a list holds multiple records.

The append() method adds another dictionary to the students list. The keys(), values(), and items() methods allow the programmer to inspect the different parts of a dictionary.

JSON Files and Comparison with CSV

JSON stands for JavaScript Object Notation. It is a readable format for saving structured information. JSON objects use curly braces and key-value pairs, while arrays use square brackets. In Python, a list of dictionaries can be saved as a JSON array of objects. Unlike CSV, which stores flat rows of comma-separated text, JSON preserves field names and can represent more complex data structures.

```
[
  {"FirstName": "Bob", "LastName": "Smith", "CourseName": "Python 100"},
  {"FirstName": "Sue", "LastName": "Jones", "CourseName": "Python 100"}
]
```

Figure 1.2. Illustrative JSON representation of two student registrations.

Reading and Writing JSON Files

Python provides the built-in json module to convert between Python collections and JSON data. json.load() reads JSON data from an open file into Python objects, while json.dump() writes Python

objects to a file. This avoids manually separating or joining fields as I did with CSV files in Assignment 04.

```
import json

with open("Enrollments.json", "r") as file:
    students = json.load(file)

with open("Enrollments.json", "w") as file:
    json.dump(students, file, indent=4)
```

Figure 1.3. General example of reading and writing a list of dictionaries with JSON.

Structured Error Handling

The try-except structure lets a program respond to anticipated errors instead of stopping unexpectedly. For example, FileNotFoundError can occur when the program tries to read a JSON file that does not yet exist. JSONDecodeError can occur when a JSON file contains invalid JSON. Handling specific exceptions makes the program easier to use and troubleshoot.

```
try:
    with open("Enrollments.json", "r") as file:
        students = json.load(file)
except FileNotFoundError:
    students = []
```

Figure 1.4. Example of handling a missing enrollment file.

Managing Code Files and GitHub

Module 05 also introduced ways to store and share code, including network locations, cloud storage, and GitHub. GitHub hosts repositories where developers can store project files and track changes. A repository can help organize Python scripts and related files and support collaboration. I still need to document any GitHub submission steps that are required for this assignment.

Creating the Program

I created an A05 project in PyCharm and placed Assignment05.py and Enrollments.json together in the project folder. The screenshot of my project also shows the .venv folder and main.py. Assignment05.py contains the course registration program, while Enrollments.json is the structured data file. The program imports json and displays a four-option menu for registering a student, showing current data, saving data, and exiting.

```
import json

MENU: str = """
--- Course Registration Program ---
1. Register a Student for a Course.
2. Show current data.
3. Save data to a file.
4. Exit the program.
"""
```

Figure 1.5. Import statement and simplified menu matching the options visible in PyCharm.

The registration workflow collects student information, creates a dictionary for each registration, and adds it to a list. The display option iterates through the list and prints each student in a readable

sentence. The save option writes the collection to Enrollments.json so that records can be used in future sessions. The complete script should be consulted for the exact implementation of each operation.

Testing the Program

I ran Assignment05.py in PyCharm. In the Run window, I selected menu option 2, "Show current data." The program displayed three registrations: Bob Smith, Sue Jones, and John Smith, all enrolled in Python 100. This confirms that the program successfully displayed the records currently available in memory.

```
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student John Smith is enrolled in Python 100
```

Figure 1.6. Student registration output observed in the PyCharm screenshot.

The PyCharm project panel showed both Assignment05.py and Enrollments.json in the A05 project. This confirmed that the two files were located together. I still need to verify the complete save-and-reload workflow by choosing option 3, exiting with option 4, restarting the program, and confirming that the same three registrations appear again. Additional exception-handling and terminal tests should also be documented after they are performed.

Summary

Module 05 helped me understand how dictionaries organize data using meaningful keys instead of positional indexes. I learned why a list of dictionaries is useful for storing multiple student records and how JSON differs from CSV. I also learned the purpose of json.load(), json.dump(), and structured error handling using try-except. Compared with Assignment 04, this assignment introduced a more descriptive way to represent student information and a structured file format for saving it. The PyCharm test confirmed that the program can display three registrations; the remaining persistence and error-handling checks will complete the testing record.

References

Introduction to Programming with Python. Module 05 – Advanced Collections and Error Handling, course notes (Canvas).

Assignment05.py. Python course registration program, November 11, 2026.

Assignment04_AvantiGulhane_FINAL.pdf. Prior knowledge document, November 4, 2026.

GitHub Repository: https://github.com/avantig86/IntroToProg-Python-Mod05